



ue

DESARROLLO DE SOFTWARE MULTIPLATAFORMA

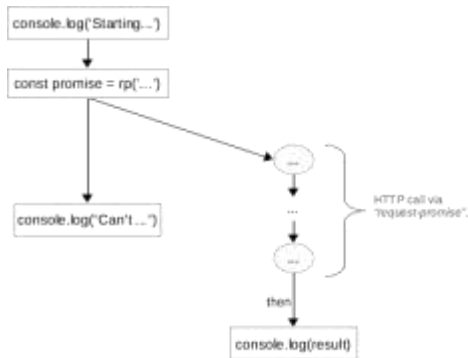
Desarrollo Web Profesional

CERVANTES ROMERO ABRAHAM

8º CUATRIMESTRE GRUPO "A"

Comunicación asíncrona accesible

Async JavaScript (JavaScript asíncrono)



Promise Chains	Async/Await
Only the promise chain itself is asynchronous	The entire wrapper function is asynchronous
Scope	
- Synchronous work can be handled in the same callback - Multiple promises use <code>Promise.all()</code>	- Synchronous work needs to be moved out of the callback - Multiple promises can be handled with simple variables
Logic	
- Then - Catch - Finally	- Try - Catch - Finally
Error Handling	

JavaScript asíncrono permite ejecutar tareas sin bloquear la ejecución principal del programa. Esto es esencial cuando se trabaja con:

- Peticiones a servidores
- Carga de datos externos
- Procesos que tardan tiempo

Los principales mecanismos son:

- **Callbacks**
- **Promesas (Promises)**
- **async/await**

Ejemplo conceptual:

- El usuario hace una acción
- Se solicita información al servidor
- La interfaz sigue funcionando mientras espera
- Cuando llega la respuesta, la UI se actualiza

Importancia:

Permite crear aplicaciones fluidas, evitando que la interfaz se “congele” mientras se cargan datos.

APIs REST



Una **API REST** es una interfaz que permite la comunicación entre cliente (frontend) y servidor (backend) mediante HTTP.

Métodos principales:

- **GET** → Obtener datos
- **POST** → Crear datos
- **PUT/PATCH** → Actualizar
- **DELETE** → Eliminar

Las respuestas suelen estar en formato JSON.

Ejemplo:

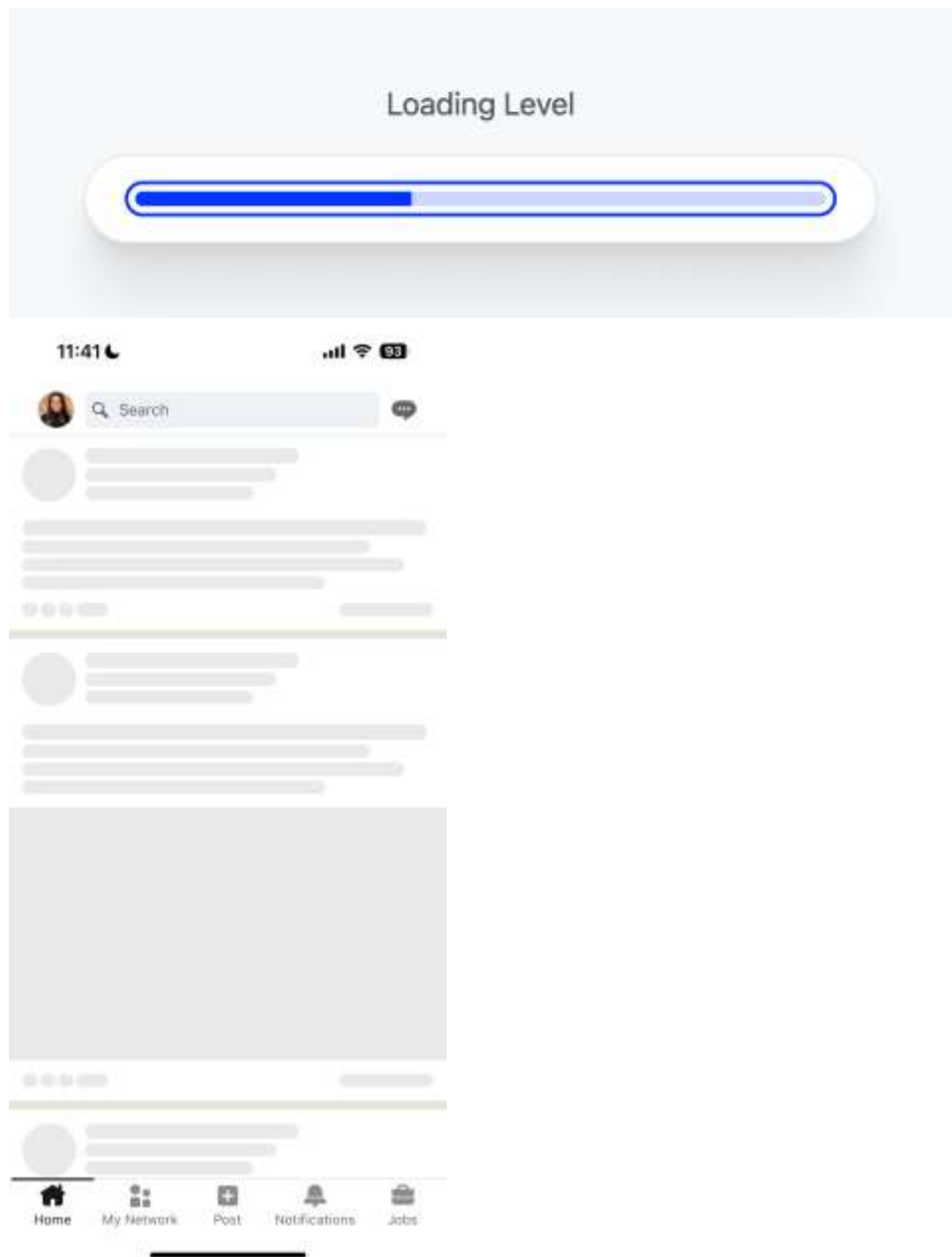
- La app solicita datos de usuarios
- El servidor responde con información
- La interfaz muestra los datos

Importancia:

Permite trabajar con datos externos y sistemas distribuidos, base de aplicaciones modernas.

Estados de carga accesibles





4

Los **estados de carga** informan al usuario que el sistema está procesando una acción.

Ejemplos:

- Spinners (círculos de carga)
- Barras de progreso
- Skeleton screens (estructura previa del contenido)

Para que sean accesibles deben:

- Indicar claramente que algo está cargando
- No depender solo de animaciones visuales
- Incluir texto como: “Cargando...”
- Usar atributos accesibles (ej. aria-live)

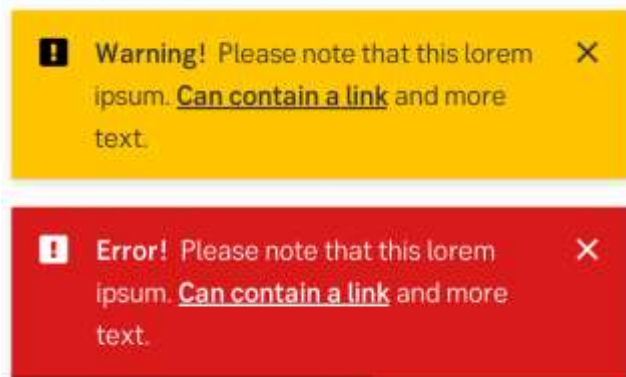
Importancia:

Evita confusión y mejora la experiencia del usuario, especialmente en conexiones lentas.

Manejo de errores de red



with icon and timeline





You're offline

Check your connection or get a notification when you're back online

[Try again](#)[Notify me](#)

Play while you wait



Hot Air Balloon

[Play](#)

Solitaire

[Play](#)

El **manejo de errores de red** se refiere a cómo el sistema responde cuando falla una petición.

Tipos de errores comunes:

- Sin conexión a internet
- Tiempo de espera (timeout)
- Error del servidor

- Datos inválidos

Buenas prácticas:

- Mostrar mensajes claros:
→ “No se pudo conectar al servidor”
- Ofrecer opciones:
→ Reintentar
- No dejar la interfaz bloqueada
- Registrar errores para diagnóstico

Importancia:

Los sistemas reales siempre fallan en algún momento. Una buena gestión de errores mejora la confianza del usuario.

¿Para qué se investiga esto?

El objetivo es que el sistema funcione correctamente en situaciones reales:

- **Con latencia:** cuando el internet es lento
- **Con fallos:** cuando el servidor no responde
- **Con datos externos:** cuando dependemos de APIs

Esto permite que la aplicación sea:

- Robusta
- Confiable
- Escalable
- Amigable para el usuario

¿Qué se debe implementar?

1 Uso de JavaScript asíncrono

- Implementar async/await o Promesas
- Evitar bloqueos en la interfaz

2 Consumo de APIs REST

- Conectar frontend con backend
- Manejar correctamente respuestas JSON

3 Estados de carga claros y accesibles

- Mostrar indicadores de carga
- Incluir texto descriptivo
- No depender solo de animaciones

4 Manejo de errores de red

- Detectar fallos
- Mostrar mensajes comprensibles
- Permitir reintentar acciones

Conclusión

La comunicación asíncrona accesible es fundamental para desarrollar aplicaciones modernas que interactúan con datos externos.

Permite que el sistema:

- Responda de forma eficiente
- Maneje fallos sin romper la experiencia
- Mantenga informado al usuario

No solo se trata de obtener datos, sino de **cómo se presentan, cómo se manejan los errores y cómo se garantiza la accesibilidad en todo momento.**